ELEMENTARY OPERATION

- We analyze an algorithm in terms of the number of elementary operations that are involved.
- An elementary operation is one whose execution time is bounded by a constant for a particular machine and programming language.
- Since the exact time for an elementary operation is unimportant, we say, that an elementary operation can be executed at unit cost.

Basic Operation:

- The operation contributing the most to the total running time of an algorithm.
- It is typically the most time consuming operation in the algorithm's innermost loop.
- We will count the number of times the algorithm's basic operation is executed on inputs of size n.

Examples

- ✓ Key comparison operation;
- ✓ Arithmetic operation (division being the most time-consuming, followed by multiplication)

FORMS OF TIME-COMPLEXITY ANALYSIS

WORST CASE

- This is the easiest form of analysis and provides an upper bound on the efficiency of an algorithm.
- Worst-case time complexity gives an upper bound on time requirements and is often easy to compute.

AVERAGE CASE

What does ‘average performance’ mean?

- Either sum the times required for every instance of a particular size, divide by the number of instances, or evaluate performance with respect to an ‘average instance’.
- **Average-case** time complexity is a less common measure:
- Average-case time is often harder to compute, and it also requires knowledge of how the input is distributed.
- Average instance-‘For a sorting algorithm this might well be a randomly ordered file.

BEST CASE

- Deals with what is the very minimum that any algorithm to solve this problem would need to do.
- Lower bound performance of the algorithm.

COMPLEXITY ESTIMATION OF SIMPLE ALGORITHMS

- We need to learn how to compare the performance different algorithms and choose the best one to solve a particular problem.
- While analyzing an algorithm, we mostly consider time complexity and space complexity.
- Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.
- Similarly, Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.
- Time and space complexity depends on lots of things like hardware, operating system, processors, etc.
- We will only consider the execution time of an algorithm.

- Time complexity estimates the time to run an algorithm.
- It's calculated by counting elementary operations.
- We often want to reason about **execution time** in a way that depends only on the **algorithm** and its **input**.
- This can be achieved by choosing an **elementary operation**, which the algorithm performs repeatedly, and define the **time complexity** T(n) as the number of such operations the algorithm performs given an array of length *n*.

```
// Compute the maximum element in the array a.

Algorithm max(a):

    max ← a[0]

    for i = 1 to len(a)-1

        if a[i] > max

            max ← a[i]

    return max
```

- For the algorithm above we can choose the comparison `a[i] > max` as an elementary operation.

1. This captures the running time of the algorithm well, since comparisons dominate all other operations in this particular algorithm.

2. Also, the time to perform a comparison is constant: it doesn't depend on the size of `a`.

- The time complexity, measured in the number of comparisons, then becomes T(*n*) = *n* - 1.
- In general, an **elementary operation** must have two properties:

1. There can't be any other operations that are performed more frequently as the size of the input grows.
2. The time to execute an elementary operation must be constant: it mustn't increase as the size of the input grows. This is known as unit cost.

Sum of 2 numbers:

```
Sum(a,b)
{
return a+b  //Takes 2 unit of time(constant)
one for arithmetic operation and one for
return.(as per above conventions)   cost=2 no
of times=1
}
```

Tsum= 2 = C =O(1)

Sum of all elements of a list :

```
list_Sum(A,n)
{//A->array and n->number of elements in the
array
total =0              // cost=1  no of times=1
for i=0 to n-1        // cost=2  no of times=n+1
(+1 for the end false condition)
sum = sum + A[i]      // cost=2  no of times=n
return sum            // cost=1  no of times=1
```

```
}
```

Tsum=1 + 2 * (n+1) + 2 * n + 1 = 4n + 4 =C1 * n + C2 = O(n)